

Kernax

User's Guide

Ziad Iziz

INFOB318 — UNamur

2026

Table des matières

1	Introduction	2
1.1	Qu'est-ce que Kernax ?	2
1.2	Pour qui ?	2
1.3	Ce que Kernax ne fait pas	2
2	Installation	3
2.1	Prérequis	3
2.2	Procédure	3
3	Quickstart	4
3.1	Code	4
4	Choisir son modèle	5
5	Fit()	6
6	Predict()	7
7	Exemples d'utilisation	8
7.1	Utiliser un Processus Gaussien	8
7.2	Utiliser un SVM	8
7.3	Utiliser la KRR	8
8	Erreurs fréquentes	9
8.1	X doit être 2D	9
8.2	Le bruit ne peut pas être négatif	9
8.3	Divergence de la loss	9

Chapitre 1

Introduction

1.1 Qu'est-ce que Kernax ?

Kernax est une bibliothèque Python d'optimisation de kernels développée par Simon Lejoly. Elle a été optimisée pour tirer pleinement partie de la compilation XLA de jax.

1.2 Pour qui ?

Kernax s'adresse aux utilisateurs recherchant une librairie de kernels rapide et exhaustive.

1.3 Ce que Kernax ne fait pas

Kernax ne prend pas en charge nativement la classification multi classes pour les SVM, entre autres. Kernax n'est aussi pas une librairie de machine learning fournissant des likelihood functions etc..

Chapitre 2

Installation

2.1 Prérequis

Kernax nécessite Python 3.10 ou plus, les dépendances principales sont Optax et JAX. L'ensemble des versions requises est listé dans requirements.txt

2.2 Procédure

La librairie peut se pip install : `pip install kernax-ml`

Chapitre 3

Quickstart

3.1 Code

```
import jax.numpy as jnp
from kernax import SEKernel, LinearKernel, WhiteNoiseKernel, ExpModule, BatchModule, A

# Create a simple Squared Exponential kernel
kernel = SEKernel(length_scale=1.0)

# Compute covariance between two points
x1 = jnp.array([1.0, 2.0])
x2 = jnp.array([1.5, 2.5])
cov = kernel(x1, x2)

# Compute covariance matrix for a set of points
X = jnp.array([[1.0], [2.0], [3.0]])
K = kernel(X, X) # Returns 3x3 covariance matrix

# Compose kernels using operators
composite_kernel = SEKernel(length_scale=1.0) + WhiteNoiseKernel(0.1) # SE + noise

# Use BatchKernel for distinct hyperparameters per batch
base_kernel = SEKernel(length_scale=1.0)
batched_kernel = BatchModule(base_kernel, batch_size=10, batch_in_axes=0, batch_over_

# Use ARDKernel for Automatic Relevance Determination
length_scales = jnp.array([1.0, 2.0, 0.5]) # Different scale per dimension
ard_kernel = ARDKernel(SEKernel(length_scale=1.0), length_scales=length_scales)
```

Chapitre 4

Choisir son modèle

- GP pour modéliser l'incertitude
- SVM pour la classification
- KRR si on veut une regression rapide sans l'incertitude

Chapitre 5

Fit()

Le comportement de fit diffère selon le modèle.

Pour le GP on passe X et les cibles Obs, le modèle optimise les hyperparamètres du kernel en maximisant le log-vraisemblance marginal via Adam puis calcule les coefficients duaux par décomposition de Cholesky. Si la loss stagne, on augmente num_iters. Si elle diverge on baisse learning_rate. NormalizeObs doit être mis à true si la moyenne des cibles n'est pas nulle.

Pour la KRR fit ne prend que les cibles, les données d'entraînement sont passées au constructeur, c'est la différence principale avec le GP.

Pour le SVM fit bascule automatiquement entre deux modes selon si un kernel est fourni ou non. Sans kernel on optimise les poids w et le biais b en minimisant la hinge loss, avec kernel le modèle passe en formulation duale et optimise les multiplicateurs de Lagrange.

Chapitre 6

Predict()

Pour le GP ce qu'on récupère dépend des arguments passés. Sans argument on obtient la moyenne postérieure, avec `Return_std=True` on récupère aussi l'écart-type, c'est utile pour visualiser l'incertitude autour des prédictions. Pour `Return_cov=True` on obtient la matrice de covariance complète. Si le modèle n'a pas encore été fitté, `predict` retourne le prior. C'est important de ne pas oublier de passer `X` en 2d.

Pour la KRR, `predict` retourne les valeurs prédites, dénormalisés si `NormalizeObs` était activé.

Pour le SVM, `predict` retourne `+1` ou `1` négatif en mode primal, et utilise les multiplieurs de Lagrange en mode dual.

Chapitre 7

Exemples d'utilisation

7.1 Utiliser un Processus Gaussien

Tout d'abord on choisit un Kernel, par exemple SE avec une length scale de 1.0. Puis on crée un objet GP en lui passant ce kernel et un niveau de noise, l'appel à fit optimise automatiquement les hyperparamètres du kernel par maximisation du log vraisemblance marginal. Une fois le modèle entraîné, predict renvoie la moyenne postérieure. C'est le modèle le plus complet des exemples.

7.2 Utiliser un SVM

Notre SVM va fonctionner en 2 modes, selon si un kernel est fourni ou non au début. Sans kernel, on opère dans un espace primal, la fit function va minimiser la hinge loss, pour trouver le poids w et le biais b . La fonction predict renvoie les classes prédites (+1 ou -1) selon la fonction $\hat{y} = \text{sign}(w^T x + b)$. Avec Kernel le modèle bascule automatiquement en formulation duale, et utilise les multiplicateurs de Lagrange. Les exemples sont disponibles dans SVMExamples.ipynb.

7.3 Utiliser la KRR

La KRR a la particularité d'avoir ses données d'entraînements passé au constructeur et non à fit. On instancie d'abord KRR(Params) puis on appelle Fit avec seulement les cibles. Le modèle résout $(K + \lambda I)\alpha = y$ par décomposition de Cholesky pour trouver les coefficients duaux et predict retourne les valeurs prédites.

Chapitre 8

Erreurs fréquentes

8.1 X doit être 2D

Les données d'entrainements doivent toujours être passé sous la forme de tableau 2D de dimensions (number of samples, number of features). Les vecteurs 1D doivent aussi être reshape.

8.2 Le bruit ne peut pas être négatif

Le noise doit aussi être strictement positif

8.3 Divergence de la loss

La fonction de loss used va dépendre du cas courant